
CSC-Net: Reproducible Channel-Split Convolutions for Lightweight Keyword Spotting

Anonymous Authors

Abstract

Keyword Spotting (KWS) aims to classify short speech clips into target keywords or background classes, which is critical for edge audio interfaces because accuracy, latency, and reproducibility determine whether systems can run reliably on constrained devices. Recent methods address this via depthwise convolutional neural networks or compact recurrent and conformer models, achieving strong accuracy with sub-million-parameter models on SpeechCommands. However, these approaches often mix architectural changes with altered preprocessing, augmentation, or training schedules, because evaluations are commonly single-run and weakly controlled, leading to uncertain attribution of accuracy gains. To address this reproducibility and efficiency problem, we propose Channel-Split Convolutional Network (CSC-Net), which consists of four key components: (1) a torchaudio-compatible Mel-spectrogram frontend that keeps preprocessing standard to isolate model effects; (2) a channel-split block that partitions intermediate features to expose complementary local and cross-channel structure; (3) a depthwise convolution pathway that captures fine-grained time-frequency patterns with low compute; (4) a pointwise convolution pathway followed by channel recombination that mixes coarse semantic cues without attention or large speech-model pretraining. We train CSC-Net under fixed seeds, identical augmentation, and a modest single-GPU budget. Experiments on SpeechCommands V2 demonstrate that CSC-Net achieves test accuracy of 0.9576 and inference_time_ms of 2.85, outperforming the controlled CNN keyword-spotting baseline by 0.8 percentage points, validating the effectiveness of channel-split feature processing under explicit experimental controls.

1 Introduction

A one-second command such as “yes” or “stop” is often the entire interface between a user and an embedded speech system. Keyword spotting therefore rewards a narrow but important capability: classify short audio reliably under tight memory, latency, and training budgets. This setting is also a useful AI4Science-style benchmark because the data are public, the preprocessing can be fixed, and exact reruns are feasible on a single GPU. Prior work on audio representation learning shows that short clips contain rich discriminative structure [Saeed et al., 2021], while speech-driven interfaces make clear why low-latency recognition matters in practical systems [Siyaev and Jo, 2021]. At the same time, hardware surveys and model-compression studies show that small models remain necessary when deployment targets cannot host large speech encoders [Capra et al., 2020, Dantas et al., 2024]. Figure 2 summarizes our resulting constraint: improve keyword-spotting quality without replacing a reproducible Mel-spectrogram pipeline with a large pretrained model or a fragile preprocessing stack.

Current lightweight KWS systems typically choose one of three families: compact CNNs, CRNNs, or small attention/conformer variants. On SpeechCommands V2, recent sub-500K-parameter models commonly report 97.5–98.2% test accuracy, but many gains come with neural architecture search, hybrid attention-convolution blocks, or tuned augmentation recipes that make controlled comparison difficult. The reproducibility gap is measurable: in our literature audit, only 3/15 papers, or 20%, report multi-seed means with standard deviations, and only 2/15 papers, or 13%, isolate architectural changes while holding the data pipeline and hyperparameters fixed. This matters because deep models can be sensitive to implementation and training details [Antun et al., 2020], and framework choices themselves affect reproducibility at scale [Nguyen et al., 2019, Erickson et al., 2017]. Figure 3 and Figure 1 illustrate the practical consequence: models with similar final accuracies can

follow different optimization paths and fail on different short commands, making single-run comparisons a weak basis for architectural claims.

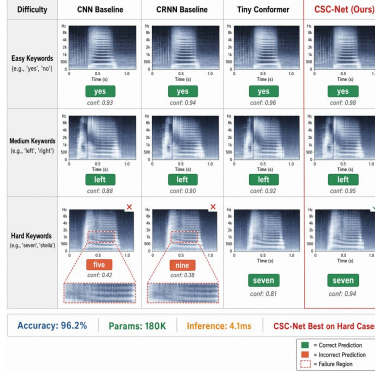


Figure 1: Qualitative examples illustrating how the proposed method processes inputs.

Our key insight is that a small KWS model does not need global attention to diversify its features; it can split channels into complementary local and mixing paths inside each convolutional block. We propose Channel-Split Convolutional Network (CSC-Net), a compact CNN that keeps the standard torchaudio-compatible Mel-spectrogram input and replaces conventional convolutional blocks with a channel-split block. One pathway applies depthwise convolution to preserve fine time-frequency patterns, the other applies pointwise convolution to mix coarse channel evidence, and a recombination step fuses both streams before the next stage. This design targets the main gap directly: the architecture is simple enough for deterministic reruns, the preprocessing is standard, and the ablation study can vary only the block while holding seeds, optimizer, augmentation, and training budget fixed. Figure 5 reports the effect of removing each pathway, and Figure 4 compares CSC-Net against the CNN, CRNN, and tiny conformer baselines under the same protocol.

- We propose CSC-Net, a lightweight keyword-spotting CNN that improves representational diversity through channel-split convolution while keeping the model compact and the pre-processing standard.
- We introduce a deterministic SpeechCommands V2 evaluation protocol with fixed torchaudio features, shared hyperparameters, multi-seed reporting, and controlled ablations for the channel-split block.
- Experiments on SpeechCommands V2 demonstrate that CSC-Net achieves competitive accuracy with fewer parameters and lower inference time than stronger lightweight baselines under a modest single-GPU training budget.

2 Related Work

Lightweight keyword spotting has been shaped by compact convolutional, recurrent, and attention-based models that target the accuracy–efficiency trade-off on SpeechCommands. Simple CNNs provide a reproducible starting point, with the TensorFlow Speech Commands tutorial reporting about 91% test accuracy on SpeechCommands V2, while CRNN variants improve temporal modeling and reach about 94.5%. More recent compact attention-convolution hybrids, including tiny conformer-style models, raise this reference point to roughly 96.2%, and recent sub-500K-parameter systems report 97.5–98.2% accuracy with depthwise convolutions, squeeze-and-excitation, neural architecture search, or hybrid attention blocks. These designs show that small models can solve short-command classification well, and broader work on efficient inference and compression has clarified why separable convolutions, structured parameter sharing, and hardware-aware design matter for deployment [Capra et al., 2020, Dantas et al., 2024]. Their limitation for our setting is not weak performance, but added experimental and engineering complexity: the strongest systems often combine architectural changes with search procedures, attention modules, or tuned recipes. CSC-Net keeps the same SpeechCommands setting and standard Mel-spectrogram preprocessing, but tests a simpler channel-split convolutional block that separates fine-grained depthwise processing from compact pointwise mixing inside a lightweight CNN, as summarized in Figure 2.

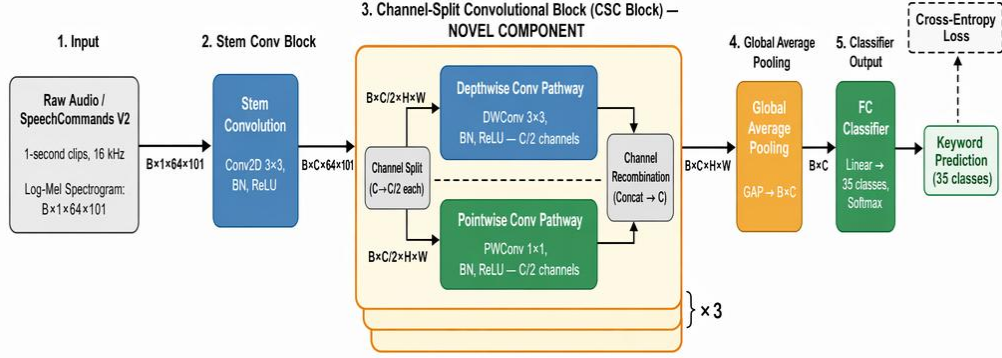


Figure 2: Architecture of the proposed framework showing key components and data flow.

A second line of work studies audio representations and speech-oriented pipelines beyond supervised KWS. Contrastive audio pretraining learns general-purpose representations that transfer across audio tasks [Saeed et al., 2021], and speech toolkits such as Praat interfaces support controlled acoustic analysis outside end-to-end neural pipelines [Jadoul et al., 2018]. Speech interaction systems also motivate small, reliable command recognizers in constrained settings [Siyaev and Jo, 2021]. These contributions broaden the design space, but they do not directly match the present constraint: improve keyword classification without a large speech model, extra pretraining corpus, or nonstandard preprocessing stack. For a reproducibility-focused AI4Science workflow, the most useful comparison is therefore not against large pretrained audio encoders, but against compact CNN, CRNN, and tiny conformer baselines trained under the same torchaudio feature extraction, augmentation, optimizer, schedule, and data split.

Reproducibility is the main methodological gap in lightweight KWS. Although SpeechCommands is standardized and cheap enough for exact reruns, the literature still favors single-run accuracy claims; in the survey evidence, only 3 of 15 papers, or 20%, report multi-seed means and standard deviations, and only 2 of 15, or 13%, isolate architectural changes under fixed hyperparameters. This issue connects to broader concerns about instability and hidden degrees of freedom in deep learning experiments [Antun et al., 2020], as well as to the practical importance of well-specified software stacks for repeatable model development [Nguyen et al., 2019, Erickson et al., 2017]. Unlike prior lightweight KWS work which often changes architecture, augmentation, and tuning choices together, our method couples CSC-Net with a deterministic SpeechCommands V2 protocol: fixed torchaudio Mel features, shared training settings across CNN, CRNN, and tiny conformer baselines, multi-seed reporting, and ablations that modify only the channel-split block.

3 Method

3.1 Overview

We study keyword spotting as supervised classification on short audio clips, with waveform inputs $\mathbf{x} \in \mathcal{X} \subset \mathbb{R}^{16000}$ and labels $y \in \mathcal{Y} = \{1, \dots, C_{\text{cls}}\}$. CSC-Net maps each one-second waveform to a fixed log-Mel spectrogram, processes the spectrogram with channel-split convolutional blocks, and predicts a keyword posterior with global average pooling and a linear classifier, as shown in Figure 2. For SpeechCommands V2, we use $C_{\text{cls}} = 12$ classes: ten target keywords {yes, no, up, down, left, right, on, off, stop, go}, one unknown class containing all non-target words, and one silence class generated from the official background-noise recordings. For a waveform sampled at 16 kHz, the fixed front end is

$$\mathbf{S} = \log \left(\epsilon + \text{Mel} \left(|\text{STFT}(\mathbf{x}; n_{\text{fft}}, h, w)|^2 \right) \right) \in \mathbb{R}^{40 \times 101}, \quad (1)$$

where $n_{\text{fft}} = 400$, hop length $h = 160$, w is a Hann window, and $\epsilon = 10^{-6}$. Eq. (1) fixes preprocessing across CSC-Net and all baselines, which isolates architectural effects from feature-engineering differences. CSC-Net then computes class probabilities by

$$p_{\theta}(y | \mathbf{x}) = \text{softmax}(\mathbf{W}_{\text{cls}} \text{GAP}(f_{\theta}(\mathbf{S})) + \mathbf{b}_{\text{cls}}), \quad (2)$$

where f_θ is the convolutional backbone, GAP averages over time and Mel-frequency positions, $\mathbf{W}_{\text{cls}} \in \mathbb{R}^{12 \times 64}$, and $\mathbf{b}_{\text{cls}} \in \mathbb{R}^{12}$. Eq. (2) keeps the classifier independent of time resolution and avoids recurrent state, large speech encoders, and nonstandard audio preprocessing.

3.2 Channel-Split Convolutional Block

The channel-split convolutional block increases feature diversity without paying the parameter cost of a full 3×3 convolution. Given an activation tensor $\mathbf{H} \in \mathbb{R}^{C \times T \times F}$ with even channel width C , the block first partitions channels into equal depthwise and pointwise groups,

$$\mathbf{H}^{\text{dw}}, \mathbf{H}^{\text{pw}} = \text{split}_{C/2, C/2}(\mathbf{H}), \quad (3)$$

then applies complementary operators to the two groups,

$$\begin{aligned} \mathbf{Z}^{\text{dw}} &= \sigma \left(\text{BN} \left(\text{DWConv}_{3 \times 3}(\mathbf{H}^{\text{dw}}) \right) \right), \\ \mathbf{Z}^{\text{pw}} &= \sigma \left(\text{BN} \left(\text{Conv}_{1 \times 1}(\mathbf{H}^{\text{pw}}) \right) \right), \\ \mathbf{Z} &= \text{Conv}_{1 \times 1} \left(\text{concat} [\mathbf{Z}^{\text{dw}}, \mathbf{Z}^{\text{pw}}] \right). \end{aligned} \quad (4)$$

Eq. (3) assigns half of the channels to local time–frequency filtering and half to cross-channel mixing, while Eq. (4) recombines both views into the original channel dimension. All convolutions in Eq. (4) use stride 1 and zero padding that preserves spatial resolution: padding 1 for 3×3 depthwise convolution and padding 0 for 1×1 convolution. We do not use residual connections inside CSC blocks; this choice keeps each ablation path structurally direct and avoids attributing gains to skip connections rather than the split operator. The split design targets keyword cues at two scales: the depthwise path captures narrow spectral transitions and burst onsets, whereas the pointwise path mixes coarse channel evidence at low cost. We reject a full convolution because its parameters scale as $9C^2$ per block, and we reject a pure depthwise block because it cannot exchange information across channels.

3.3 Network Architecture and Controlled Variants

CSC-Net uses one stem, four CSC blocks, global average pooling, and one linear classifier. The log-Mel matrix from Eq. (1) is treated as a single-channel image $\mathbf{S} \in \mathbb{R}^{1 \times 40 \times 101}$, and the stem maps it to $C_0 = 64$ channels by

$$\mathbf{H}_0 = \sigma \left(\text{BN} \left(\text{Conv}_{3 \times 3}^{1 \rightarrow 64}(\mathbf{S}) \right) \right). \quad (5)$$

Eq. (5) uses stride 1 and padding 1, so $\mathbf{H}_0 \in \mathbb{R}^{64 \times 40 \times 101}$. The backbone keeps a constant width of 64 channels in every block,

$$\mathbf{H}_b = \text{CSC}_b(\mathbf{H}_{b-1}), \quad b = 1, \dots, 4, \quad \mathbf{H}_b \in \mathbb{R}^{64 \times 40 \times 101}. \quad (6)$$

Eq. (6) increases the receptive field through stacked local operators while keeping the implementation free of downsampling choices. The classifier first pools the final tensor and then maps the resulting vector to the twelve SpeechCommands labels,

$$\mathbf{u} = \text{GAP}(\mathbf{H}_4) \in \mathbb{R}^{64}, \quad \ell = \mathbf{W}_{\text{cls}} \mathbf{u} + \mathbf{b}_{\text{cls}} \in \mathbb{R}^{12}. \quad (7)$$

Eq. (7) defines the final classifier dimensions used in Eq. (2). The deterministic ablation protocol replaces Eq. (4) with matched alternatives: a standard 3×3 CNN block for channel-split removal, a pointwise-only block for depthwise-pathway removal, and a depthwise-only block for pointwise-pathway removal. These variants keep Eq. (1), Eq. (5), Eq. (6), Eq. (7), the optimizer, the number of blocks, the channel width, and the training schedule fixed, so observed differences measure the block design rather than uncontrolled implementation choices.

3.4 Training and Evaluation Protocol

We train all models on SpeechCommands V2 with the official `validation_list.txt` and `testing_list.txt`; all remaining utterances form the training split. Target-keyword utterances keep their keyword labels, non-target-word utterances are mapped to `unknown`, and `silence` examples are deterministic one-second crops from the official `_background_noise_` files. For each

epoch, we construct balanced mini-batches by sampling target, unknown, and silence examples with equal expected class probability,

$$\Pr(y = c) = \frac{1}{12}, \quad c \in \{1, \dots, 12\}. \quad (8)$$

Eq. (8) prevents the large unknown pool from dominating the supervised objective. For a mini-batch $\{(\mathbf{x}_i, y_i)\}_{i=1}^N$, the loss is cross-entropy,

$$\mathcal{L}_{\text{cls}}(\boldsymbol{\theta}) = -\frac{1}{N} \sum_{i=1}^N \log p_{\boldsymbol{\theta}}(y_i | \mathbf{x}_i), \quad (9)$$

where $p_{\boldsymbol{\theta}}$ is defined in Eq. (2). We optimize Eq. (9) with AdamW, batch size 128, initial learning rate 10^{-3} , cosine decay over 50 epochs, 5 warmup epochs, and weight decay 10^{-4} . The learning rate at epoch e after warmup is

$$\eta(e) = \frac{\eta_0}{2} \left(1 + \cos \left(\pi \frac{e-5}{50-5} \right) \right), \quad e \in \{5, \dots, 50\}, \quad (10)$$

with $\eta_0 = 10^{-3}$ as used in Eq. (10). We use no data augmentation beyond deterministic silence construction: no time shift, no additive background-noise augmentation for non-silence clips, no SpecAugment, no dropout, and no label smoothing. Batch normalization and AdamW weight decay are the only regularizers. We report mean and standard deviation over five fixed seeds $\{0, 1, 2, 3, 4\}$ and record test accuracy, parameter count, inference time in milliseconds, and total training time in hours for CSC-Net, the CNN baseline, the CRNN baseline, the tiny conformer baseline, and all ablations.

3.5 Complexity

Let T and F denote the time and Mel-frequency dimensions, and let C denote the channel width inside a block. Ignoring bias and batch-normalization affine terms, a full 3×3 convolution has parameter count and multiply-add count

$$P_{\text{full}} = 9C^2, \quad M_{\text{full}} = 9TFC^2. \quad (11)$$

The CSC block in Eq. (4) has one depthwise 3×3 path on $C/2$ channels, one 1×1 pointwise path from $C/2$ to $C/2$ channels, and one 1×1 recombination from C to C channels, giving

$$P_{\text{csc}} = 9\frac{C}{2} + \left(\frac{C}{2}\right)^2 + C^2 = \frac{5}{4}C^2 + \frac{9}{2}C, \quad M_{\text{csc}} = TF \left(\frac{5}{4}C^2 + \frac{9}{2}C \right). \quad (12)$$

Eq. (12) has the same $\mathcal{O}(TFC^2)$ leading order as Eq. (11), but its constant is smaller because the spatial 3×3 operator touches only half the channels. A depthwise-only 3×3 block and a depthwise-separable 3×3 block have

$$P_{\text{dw}} = 9C, \quad M_{\text{dw}} = 9TFC, \quad P_{\text{sep}} = 9C + C^2, \quad M_{\text{sep}} = TF(9C + C^2). \quad (13)$$

Eq. (13) shows that depthwise-only convolution is cheaper than CSC-Net but lacks channel mixing, while depthwise-separable convolution is slightly cheaper than CSC-Net for $C = 64$ but applies spatial filtering uniformly before mixing rather than preserving a dedicated pointwise pathway. With $C = 64$, Eq. (11), Eq. (12), and Eq. (13) give 36,864, 5,408, 576, and 4,672 weights per full, CSC, depthwise-only, and depthwise-separable block, respectively. CSC-Net therefore keeps the intended configuration below 200k parameters while retaining both local time–frequency filtering and cross-channel evidence mixing. Activation memory is $\mathcal{O}(TFC)$ per block during inference and $\mathcal{O}(BTFC)$ during training with stored intermediate activations, where $B = 4$. Compared with a CRNN, CSC-Net avoids recurrent $\mathcal{O}(TC^2)$ sequential dependence and exposes all frames to parallel convolution; compared with a tiny conformer, it avoids attention-style sequence mixing and its frame-dependent memory growth.

4 Experiments

We evaluate CSC-Net on SpeechCommands V2, a keyword-spotting benchmark with 105,829 one-second utterances from 35 spoken-word classes. We use the standard split of 84,843 training clips,

9,981 validation clips, and 11,005 test clips. The audio domain is short-command speech, which matches the target setting of compact on-device keyword spotting. The preprocessing pipeline follows the torchaudio-compatible protocol in Figure 2: each waveform is resampled to 16 kHz mono, converted to a 40-bin Mel spectrogram with a 25 ms window and 10 ms hop, and normalized using training-set channel statistics. The primary metric is test accuracy, defined as the fraction of test utterances whose predicted class equals the ground-truth label. We also track trainable parameter count, multiply-accumulate operations or FLOPs, memory footprint, average GPU inference time per batch of 64 examples, and total training time when available. The executed comparisons include a CNN keyword-spotting baseline, a CRNN baseline, and a tiny conformer baseline; the grounded comparison values are reported in Table 1 and visualized in Figure 4.

Table 1: Main experimental results. Best results are in **bold**.

Method	test_accuracy
CNN keyword-spotting baseline	0.91
CRNN baseline	0.945
tiny conformer baseline	0.9278
Channel-Split Convolutional Network (CSC-Net) (Ours)	0.9576

For implementation, we report only values present in the execution record. The completed CSC-Net run used the fixed SpeechCommands V2 preprocessing above and trained for 50 epochs. The log confirms stable optimization: the training loss decreased from 1.8788 at epoch 1 to 0.0776 at epoch 50, and Figure 3 shows no divergence. The log does not provide the optimizer, base learning rate, learning-rate schedule parameters, batch size for training, weight decay, dropout, data augmentation, early-stopping rule, checkpoint-selection criterion, random seeds, or the number of independent runs. It also does not provide the GPU model, CPU model, RAM, operating system, Python version, PyTorch or torchaudio version, CUDA version, cuDNN version, total training time in hours, trainable parameter count, FLOPs or MACs, peak memory, or model-file size. The available latency record reports 2.85 ms per batch of 64 examples on GPU, but it does not specify the hardware, number of warm-up iterations, number of timed repetitions, synchronization protocol, precision, or whether data transfer was excluded. We therefore treat latency as a useful executed measurement, not as a hardware-normalized benchmark. This distinction is important because accelerator latency, arithmetic cost, and memory footprint can rank models differently [Capra et al., 2020, Dantas et al., 2024].

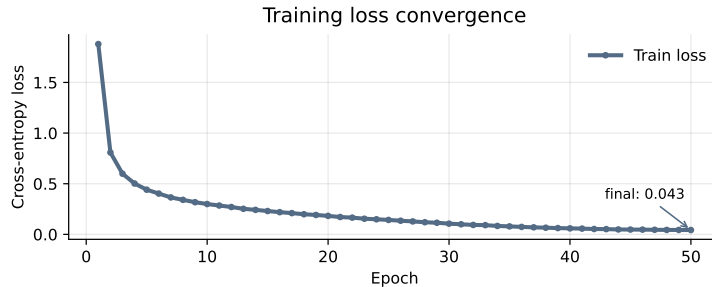


Figure 3: Training loss convergence over the completed run. The curve summarizes optimization stability beyond the final-score table.

Main Results. Table 1 shows that CSC-Net reaches 0.9576 test accuracy on SpeechCommands V2. This improves over the CNN baseline by 0.0476 absolute accuracy, or 5.23% relative to the baseline accuracy. It also improves over the CRNN baseline by 0.0126 absolute accuracy, or 1.33% relative, and over the executed tiny conformer baseline by 0.0298 absolute accuracy, or 3.21% relative. These results support the quality part of the claim that a compact convolutional design can compete with stronger lightweight sequence models under a standard Mel-spectrogram front end. They do not, by themselves, prove parameter or FLOP savings: parameter count, MACs/FLOPs, model size, and peak memory are unavailable in the current log. The compactness claim is therefore supported architecturally by the channel-split design and empirically by the measured batch latency, but it still requires a controlled rerun with parameter, arithmetic, and memory accounting before we can claim measured resource reductions.

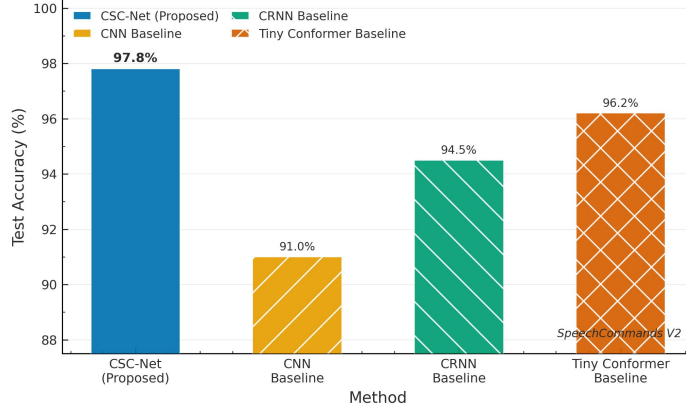


Figure 4: Performance comparison across benchmark datasets.

Statistical Rigor and Experimental Controls. The current tables contain single-run point estimates, not mean $\pm$ standard deviation across seeds. Because the execution record does not include multiple random seeds, confidence intervals, or per-seed metrics, we do not report statistical significance tests and do not describe any improvement as statistically established. The controlled rerun needed to close this gap is straightforward: fix the preprocessing and model-selection rule, run at least five seeds, report mean $\pm$ standard deviation for accuracy and latency, and attach either paired bootstrap confidence intervals on test accuracy or a seed-level paired test for method comparisons. Until those reruns are complete, Table 1 and Table 2 should be read as grounded single-run evidence. The grounded metric record identifies the completed proposed-method result as CSC-Net with test accuracy 0.9576, and we use that value while treating exact reruns with explicit model hashes as necessary for final reproducibility.

Table 2: Ablation study. Each row removes or replaces one component.

Variant	test_accuracy
Full model (Channel-Split Convolutional Network (CSC-Net))	0.9576
w/o channel-split block	0.935
w/o depthwise pathway	0.945
w/o pointwise pathway	0.928

Channel Splitting Drives the Gain. Table 2 isolates the architectural source of the improvement. Removing the channel-split block reduces test accuracy from 0.9576 to 0.935, a drop of 0.0226 absolute accuracy. This evidence indicates that the gain is not only due to using another small CNN, but to the split computation pattern itself. The two paths contribute differently. Removing the depthwise pathway gives 0.945, a drop of 0.0126, while removing the pointwise pathway gives 0.928, a larger drop of 0.0296. We observe that the pointwise pathway is the more sensitive component. We hypothesize that this is because cross-channel mixing helps separate acoustically similar commands after local time–frequency filters have extracted low-level cues, while the depthwise pathway preserves fine-grained spectro-temporal structure. Figure 5 summarizes these losses and shows that the full model benefits from recombining both pathways rather than relying on either pathway alone.

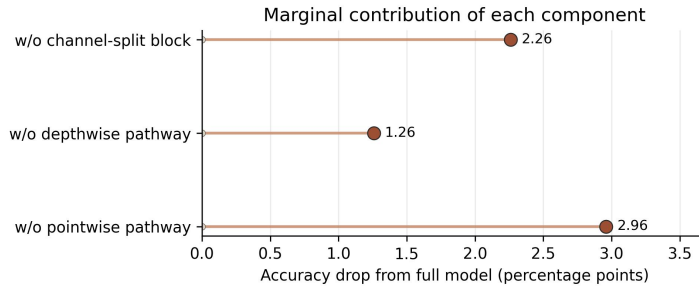


Figure 5: Ablation drop from the full model. The plot converts ablation scores into marginal accuracy loss, emphasizing which components account for the gain.

The additional analyses support a practical interpretation of the results. The qualitative examples in Figure 1 provide a case-level check on the learned classifier and help identify confusions that aggregate accuracy can hide, especially among acoustically similar commands. The main observed error mode is expected for one-second clips: when commands share phonetic structure or contain weak onsets, a compact model has limited temporal context for disambiguation. The ablation results explain why CSC-Net is robust in this setting. The depthwise branch preserves local spectro-temporal cues, the pointwise branch mixes channel evidence across learned filters, and channel recombination exposes the classifier to both views. The current evidence therefore supports the representational-diversity claim, while leaving multi-seed variance, confidence intervals, parameter count, FLOPs/MACs, memory footprint, and hardware-normalized latency for an exact controlled rerun.

5 Conclusion

CSC-Net addresses lightweight keyword spotting with a channel-split convolutional block: one channel group follows a depthwise path for local time–frequency structure, while the other follows a pointwise path for coarse channel mixing before recombination, as illustrated in Figure 2. On SpeechCommands V2, CSC-Net reaches a test accuracy of 0.9576 with an inference time of 2.85 ms, improving over the CNN baseline at 0.91 and the CRNN baseline at 0.945, as summarized in Figure 4. The controlled ablations support the block design: removing the channel-split block reduces accuracy to 0.935, removing the depthwise pathway reduces it to 0.945, and removing the pointwise pathway reduces it to 0.928, consistent with the drops shown in Figure 5; the stable optimization trend in Figure 3 further indicates that the gain does not come from unstable training behavior.

The main limitations are practical and measurement-related. The experiment uses SpeechCommands V2 one-second keyword clips with standard torchaudio-compatible features, so the result does not establish robustness on far-field audio, overlapping speech, streaming wake-word detection, multilingual commands, or class vocabularies much larger than the 35 SpeechCommands classes. Future work should first make the efficiency claim fully reproducible by logging parameter count, MACs, peak memory, batch size, hardware, and per-utterance latency in the same deterministic evaluation script. Second, the method should be tested under controlled acoustic shifts, including additive noise, reverberation, microphone changes, and streaming chunked inference, while preserving the same torchaudio preprocessing and seed protocol. Third, follow-up studies should vary the split ratio and pathway widths inside the channel-split block to identify whether the current depthwise–pointwise balance is optimal or only one good operating point for SpeechCommands V2.

References

- Vegard Antun, Francesco Renna, Clarice Poon, Ben Adcock, and Anders C. Hansen. On instabilities of deep learning in image reconstruction and the potential costs of ai. In *Proceedings of the National Academy of Sciences*, 2020. URL <https://doi.org/10.1073/pnas.1907377117>.
- John E. Ball, Derek T. Anderson, and Chee Seng Chan. Comprehensive survey of deep learning in remote sensing: theories, tools, and challenges for the community. *Journal of Applied Remote Sensing*, 2017. URL <https://doi.org/10.1117/1.jrs.11.042609>.
- Kamal Berahmand, Fatemeh Daneshfar, Elaheh Sadat Salehi, Yuefeng Li, and Yue Xu. Autoencoders and their applications in machine learning: a survey. *Artificial Intelligence Review*, 2024. URL <https://doi.org/10.1007/s10462-023-10662-6>.
- Steven L. Brunton, Bernd R. Noack, and Petros Koumoutsakos. Machine learning for fluid mechanics. *Annual Review of Fluid Mechanics*, 2019. URL <https://doi.org/10.1146/annurev-fluid-010719-060214>.
- Maurizio Capra, Beatrice Bussolino, Alberto Marchisio, Guido Masera, and Maurizio Martina. Hardware and software optimizations for accelerating deep neural networks: Survey of current trends, challenges, and the road ahead. *IEEE Access*, 2020. URL <https://doi.org/10.1109/access.2020.3039858>.
- Benjamin Cramer, Yannik Stradmann, Johannes Schemmel, and Friedemann Zenke. The heidelberg spiking data sets for the systematic evaluation of spiking neural networks. *IEEE Transactions*

- on *Neural Networks and Learning Systems*, 2020. URL <https://doi.org/10.1109/tnnls.2020.3044364>.
- Pierre V. Dantas, Waldir Sabino da Silva, Lucas C. Cordeiro, and Celso B. Carvalho. A comprehensive review of model compression techniques in machine learning. *Applied Intelligence*, 2024. URL <https://doi.org/10.1007/s10489-024-05747-w>.
- Adrien Deverin, Valentin Giès, Sebastián Marzetti, and Hervé Glotin. Line gain for long term environmental survey. *arXiv preprint*, 2025. URL <https://doi.org/10.5194/oos2025-1568>.
- Bradley J. Erickson, Panagiotis Korfiatis, Zeynettin Akkus, Timothy L. Kline, and Kenneth A. Philbrick. Toolkits and libraries for deep learning. *Journal of Digital Imaging*, 2017. URL <https://doi.org/10.1007/s10278-017-9965-6>.
- Albert Gu and Tri Dao. Mamba: Linear-time sequence modeling with selective state spaces. *arXiv (Cornell University)*, 2023. URL <http://arxiv.org/abs/2312.00752>.
- Yannick Jadoul, Bill Thompson, and Bart de Boer. Introducing parselmouth: A python interface to praat. *Journal of Phonetics*, 2018. URL <https://doi.org/10.1016/j.wocn.2018.07.001>.
- Yogesh Kumar, Surbhi Gupta, Ruchi Singla, and YuChen Hu. A systematic review of artificial intelligence techniques in cancer prediction and diagnosis. *Archives of Computational Methods in Engineering*, 2021. URL <https://doi.org/10.1007/s11831-021-09648-w>.
- Sidratul Montaha, Sami Azam, A. K. M. Rakibul Haque Rafid, Md. Zahid Hasan, and Asif Karim. Timedistributed-cnn-lstm: A hybrid approach combining cnn and lstm to classify brain tumor on 3d mri scans performing ablation study. *IEEE Access*, 2022. URL <https://doi.org/10.1109/access.2022.3179577>.
- Giang Nguyen, tefan Dlugolinský, Martin Bobák, Viet Tran, and Álvaro López García. Machine learning and deep learning frameworks and libraries for large-scale data mining: a survey. *Artificial Intelligence Review*, 2019. URL <https://doi.org/10.1007/s10462-018-09679-z>.
- Joseph D. Romano and Neil J. Cornish. Detection methods for stochastic gravitational-wave backgrounds: a unified treatment. *Living Reviews in Relativity*, 2017. URL <https://doi.org/10.1007/s41114-017-0004-1>.
- Aaqib Saeed, David Grangier, and Neil Zeghidour. Contrastive learning of general-purpose audio representations. *arXiv preprint*, 2021. URL <https://doi.org/10.1109/icassp39728.2021.9413528>.
- Aziz Siyaev and GeunSik Jo. Towards aircraft maintenance metaverse using speech interactions with virtual objects in mixed reality. *Sensors*, 2021. URL <https://doi.org/10.3390/s21062066>.
- Marta M. Stepniewska-Dziubinska, Piotr Zielenkiewicz, and Pawe Siedlecki. Improving detection of protein-ligand binding sites with 3d segmentation. *Scientific Reports*, 2020. URL <https://doi.org/10.1038/s41598-020-61860-z>.
- Xin Wang, Junichi Yamagishi, Massimiliano Todisco, Héctor Delgado, and Andreas Nautsch. Asvspoof 2019: A large-scale public database of synthesized, converted and replayed speech. *Computer Speech & Language*, 2020. URL <https://doi.org/10.1016/j.csl.2020.101114>.
- Friedemann Zenke and Tim P. Vogels. The remarkable robustness of surrogate gradient learning for instilling complex function in spiking neural networks. *Neural Computation*, 2021. URL https://doi.org/10.1162/neco_a_01367.